



Visão Computacional e Deep Learning

Ma. Cilene Renata Real
cilene.real@facens.br

Apresentação da Atividade 1

As duplas, deverão realizar a apresentação da sua atividade, a partir da metodologia de apresentação selecionada.

Sistema de coordenadas e manipulação de pixels

Imagens preto e branca são compostas de apenas **uma matriz de duas dimensões** como por exemplo:

	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255
	15	30	45	60	75	90	105	120	135	150	165	180	195	210	225	240	255

A imagem representada em uma matriz de inteiros onde cada célula é um inteiro sem sinal de 8 bits que pode conter de 0 (preto) até 255 (branco).

Observe os vários tons de cinza nos valores intermediários como 30 (cinza escuro) e 210 (cinza claro).

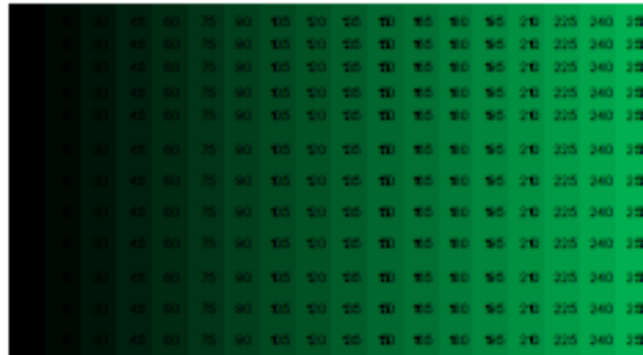
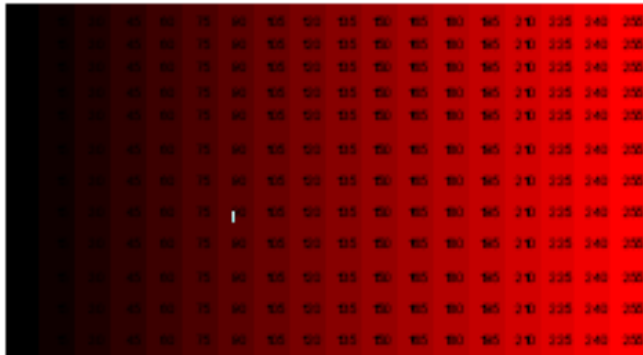
Sistema de coordenadas e manipulação de pixels

O sistema de coordenadas da imagem branco e preto, possui apenas duas posições: linha, coluna. Os índices iniciam em 0.

Já em uma imagem com 300 pixels de largura, ou seja, 300 colunas e tendo 200 pixels de altura, ou seja, 200 linhas, terá o pixel (199,299).

Sistema de coordenadas e manipulação de pixels

Já para imagens coloridas temos três dessas matrizes de duas dimensões cada uma representando uma das cores do sistema RGB:



Portanto, cada pixel é formado de uma tupla de 3 inteiros de 8 bits sem sinal no sistema (R,G,B).

As cores mais comuns são:

Branco - RGB (255,255,255);

Azul - RGB (0,0,255);

Vermelho - RGB (255,0,0);

Verde - RGB (0,255,0);

Amarelo - RGB (255,255,0);

Magenta - RGB (255,0,255);

Ciano - RGB (0,255,255);

Preto - RGB (0,0,0).

Para realizar a parte prática do sistema de coordenadas utilizaremos:

- Colab
- Python
- OpenCV

OpenCV (Open Source Computer Vision Library)

É uma biblioteca multiplataforma que fornece mais de 2500 algoritmos otimizados para visão computacional e processamento de imagens.

Desenvolvida inicialmente pela Intel em 1999, hoje é mantida por uma comunidade ativa de desenvolvedores.

A biblioteca oferece funções para tarefas como:

- Manipulação básica de imagens (carregamento, salvamento, transformações)
- Processamento (filtragem, morfologia, transformações)
- Análise (histogramas, estatísticas, detecção de características)
- Detecção e reconhecimento de objetos
- Operações com vídeo e calibração de câmera

Para essa atividade, será necessário uma imagem de vocês em formato jpg, e devemos chamá-la de entrada.jpg

Criem uma pasta no Google Drive, com o nome de Visao, por exemplo e insiram essa imagem.

Vamos abrir o colab, renomeá-lo para aula1 e compartilhá-lo, pois, essa atividade será enviada.

No colab, iniciem criando uma Caixa de texto com o nome de exemplos:

Exemplos

No primeiro bloco de códigos, vamos inserir as configurações iniciais:

```
#Configurações Iniciais
import pandas as pd #importa a biblioteca pandas, que é essencial para manipulação e análise de dados em Python.
from google.colab import drive # importa a função drive do módulo google.colab
drive.mount('/content/drive') #conectar seu Google Drive ao ambiente do Colab

import os #importa o módulo os
os.chdir("/content/drive/") # muda o diretório de trabalho atual para /content/drive/
!ls # exibirá todos os arquivos e pastas que estão diretamente dentro do seu Google Drive montado.
```

+Código:

```
import cv2
from google.colab.patches import cv2_imshow # Importa a função patched para Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

# Verifica se a imagem foi carregada corretamente
if imagem is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    print('Largura em pixels: ', end='')
    print(imagem.shape[1]) #largura da imagem
    print('Altura em pixels: ', end='')
    print(imagem.shape[0]) #altura da imagem
    print('Qtde de canais: ', end='')
    print(imagem.shape[2])

    cv2_imshow(imagem) # Usa cv2_imshow para exibir a imagem no Colab
    # cv2.waitKey(0) # waitKey e destroyAllWindows não são necessários com cv2_imshow
    # cv2.destroyAllWindows()

    cv2.imwrite("saida.jpg", imagem)
```

+Código:

```
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

if imagem is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    for y in range(0, imagem.shape[0]):
        for x in range(0, imagem.shape[1]):
            imagem[y, x] = (255,0,0) # Define o pixel como azul (BGR)

    cv2_imshow(imagem) # Exibe a imagem modificada apenas uma vez após todas as alterações
```

A imagem é lida e armazenada em 'imagem' que é uma variável que dará acesso ao objeto da imagem que nada mais é que uma matriz de 3 dimensões (3 canais) contendo em cada dimensão uma das 3 cores do padrão RGB (red=vermelho, green-verde, blue=azul).

No caso de uma imagem preto e branca temos apenas um canal, ou seja, apenas uma matriz de 2 dimensões.

+Código:

```
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

if imagem is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    for y in range(0, imagem.shape[0]): #percorre linhas
        for x in range(0, imagem.shape[1]): #percorre colunas
            imagem[y, x] = (x%256,y%256,x%256)

    cv2_imshow(imagem) # Display the image once after all modifications
    # cv2.waitKey(0) and cv2.destroyAllWindows() are not needed with cv2_imshow in Colab
```

+Código:

```
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

if imagem is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    for y in range(0, imagem.shape[0], 1): #percorre as linhas
        for x in range(0, imagem.shape[1], 1): #percorre as colunas
            imagem[y, x] = (0, (x*y)%256, 0) # Define o pixel com um padrão de cor baseado em x*y

cv2_imshow(imagem) # Exibe a imagem modificada apenas uma vez após todas as alterações
# cv2.waitKey(0) and cv2.destroyAllWindows() are not needed with cv2_imshow in Colab
```

+Código:

```
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg') # Corrected filename

if imagem is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    for y in range(0, imagem.shape[0], 10): #percorre linhas
        for x in range(0, imagem.shape[1], 10): #percorre colunas
            imagem[y:y+5, x: x+5] = (0,255,255) # Define um bloco de 5x5 pixels como ciano (BGR)

cv2_imshow(imagem) # Exibe a imagem modificada apenas uma vez após todas as alterações
# cv2.waitKey(0) and cv2.destroyAllWindows() are not needed with cv2_imshow in Colab
```


+Código:

```
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

image = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')
image[30:50, :] = (255, 0, 0) #azul
image[100:150, 50:100] = (0, 0, 255) #vermelho
image[60:100, 200:220] = (0, 255, 255) #amarelo
image[150:300, 250:350] = (0, 255, 0) #verde
image[300:400, 50:150] = (255, 255, 0) #ciano
image[250:350, 300:400] = (255, 255, 255) #branco
image[70:100, 300: 450] = (0, 0, 0) #preto
cv2_imshow(image) # Use cv2_imshow instead of cv2.imshow
cv2.imwrite("alterada.jpg", image)
# cv2.waitKey(0) is not needed with cv2_imshow
```

+Código:

```
import numpy as np
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')
vermelho = (0, 0, 255)
verde = (0, 255, 0)
azul = (255, 0, 0)
cv2.line(imagem, (0, 0), (100, 200), verde)
cv2.line(imagem, (300, 200), (150, 150), vermelho, 5)
cv2.rectangle(imagem, (20, 20), (120, 120), azul, 10)
cv2.rectangle(imagem, (200, 50), (225, 125), verde, -1)

(X, Y) = (imagem.shape[1] // 2, imagem.shape[0] // 2)
for raio in range(0, 175, 15):
    cv2.circle(imagem, (X, Y), raio, vermelho)
cv2_imshow(imagem) # Use cv2_imshow for Colab
```

+Código:

```
import numpy as np
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

# Check if image loaded correctly
if imagem is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    fonte = cv2.FONT_HERSHEY_SIMPLEX
    cv2.putText(imagem, 'Primeira aula de Visao Computacional', (15, 65), fonte, 2, (255, 255, 255), 2, cv2.LINE_AA)
    cv2_imshow(imagem) # Use cv2_imshow for Colab
```

+Código:

```
#Cortando a imagem
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

imagem = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

# Check if image loaded correctly
if imagem is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    # Correcting the slicing: start index must be less than end index
    recorte = imagem[200:500, 200:500] # Example: cutting a 300x300 pixel region
    cv2_imshow(recorte) # Use cv2_imshow for Colab
    cv2.imwrite("recorte.jpg", recorte) #salva no disco
```

+Código:

#Redimensionando a imagem

```
import numpy as np
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

img = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

# Check if image loaded correctly
if img is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
else:
    # cv2.imshow("Original", img) # Disabled in Colab
    cv2_imshow(img) # Use cv2_imshow for Colab to display original

    largura = img.shape[1]
    altura = img.shape[0]
    proporcao = float(altura/largura)
    largura_nova = 320 #em pixels
    altura_nova = int(largura_nova*proporcao)
    tamanho_novo = (largura_nova, altura_nova)
    img_redimensionada = cv2.resize(img, tamanho_novo, interpolation = cv2.INTER_AREA)
    # cv2.imshow('Resultado', img_redimensionada) # Disabled in Colab
    cv2_imshow(img_redimensionada) # Use cv2_imshow for Colab to display resized
```

+Código:

```
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

img = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

# Check if image loaded correctly
if img is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
elif img.size == 0:
    print("Erro: A imagem 'entrada.jpg' está vazia ou corrompida.")
else:
    cv2_imshow(img)

    flip_horizontal = img[::-1,:] #comando equivalente abaixo
    cv2_imshow(flip_horizontal)

    flip_vertical = img[:,::-1] #comando equivalente abaixo
    cv2_imshow(flip_vertical)

    flip_hv = img[::-1,::-1] #comando equivalente abaixo
    cv2_imshow(flip_hv)
```

+Código:

```
import cv2
from google.colab.patches import cv2_imshow # Import the patched function for Colab

img = cv2.imread('/content/drive/My Drive/Visao/entrada.jpg')

# Check if image loaded correctly
if img is None:
    print("Erro: Não foi possível carregar a imagem 'entrada.jpg'. Verifique o caminho e se o arquivo existe.")
elif img.size == 0:
    print("Erro: A imagem 'entrada.jpg' está vazia ou corrompida.")
else:
    (alt, lar) = img.shape[:2] # captura altura e largura
    centro = (lar // 2, alt // 2) # acha o centro
    M = cv2.getRotationMatrix2D(centro, 30, 1.0) # 30 graus
    img_rotacionada = cv2.warpAffine(img, M, (lar, alt))
    cv2_imshow(img_rotacionada)
```

Exercícios



Dúvidas?

MUITO OBRIGADA!!!!



cilene.real@facens.br